

Teknisk Dokumentation



Examensarbete | Yotaka Khaowkomol
CLO24– Campus Mölndal
HT 2025

https://github.com/Bombalaka/Random_Menu

Teknisk Dokumentation	3
1. Inledning	3
2. Produkten	4
2.1 Översikt	4
2.3 Arkitekturval	5
3. Verktyg och Teknologier	7
Frontend	7
Backend	8
4.1 Systemöversikt	10
4.2 Fil- och mappstruktur	11
Frontend-struktur (React Native Application)	11
Backendstruktur	14
5. Applikationens Komponenter	16
Backend componenter	17
Övriga Lambda-funktioner	20
Databas	21
Frontend componenter	22
Interna Moduler	25
6. Flöden	27
Autentisering	28
Dataflöden	28
Externa API:er	30
7. Testning	32
Testningsstrategi	32
Testmetodik	32
Testresultat	33
8. Deployment	33
Körning och Deployment	34
Miljövariabler och Konfiguration	34
Future Enhancements	35
9. Referenser	37

Teknisk Dokumentation

1. Inledning

Detta projekt skapades som den avslutande uppgiften i Cloud Development för att demonstrera de kunskaper jag har lärt mig under kursens gång. Jag ville också skapa en lösning på ett verkligt problem och visa hur teknik kan stödja vardagliga situationer. För backend valde jag att använda serverless-arkitektur med API Gateway och NoSQL-databasen DynamoDB. För frontend byggde jag applikationen med React Native och ramverket Expo.

Applikationen fokuserar på huvudfunktionen: att generera menyer baserat på användarens favoriträtter för att hjälpa till med den dagliga kampen att bestämma vad man ska äta. Dessutom finns en AI-driven förslagsfunktion som arbetar i bakgrunden för att ge mer variation på maträtter. Användare kan hantera sina favoritmenyer med full CRUD-funktionalitet (Create, Read, Update, Delete), vilket ger dem kontroll att redigera sin personliga menylista när de vill.

Tekniska mål:

- Lära mig och implementera AWS serverless-tjänster (Lambda, API Gateway, DynamoDB)
- Bygga och driftsätta ett RESTful API
- Utveckla en plattformsoberoende mobilapplikation med React Native
- Integrera AI-tjänster med AWS Bedrock
- Praktisera god molnarkitektur
- Förstå enhetsbaserad autentisering

2. Produkten

2.1 Översikt

Backenden inkluderar 9 Lambda-funktioner som hanterar olika operationer: CRUD-operationer för mathantering, menygenerering, AI-drivna förslag och receptgenerering. Systemet använder AWS Bedrock för AI-integration, och utnyttjar förtränade modeller med anpassade prompter för att generera personaliserade rekommendationer. Data lagras i DynamoDB med separata tabeller för maträtter, enhetsregistreringar och förslagshistorik. Alla komponenter kommunicerar genom AWS API Gateway, som routar förfrågningar mellan mobilfrontend och backend Lambda-funktioner.

2.2 Problem som löses

Under utvecklingen av detta projekt identifierades flera tekniska utmaningar som framgångsrikt hanterades:

1. **Serverless Backend Architecture**

Att designa en händelsedriven backend med AWS Lambda och API Gateway tog bort behovet av traditionella servrar. Jag säkerställde åtkomstkontroll genom IAM-policys och aktiverade automatisk skalbarhet så att systemet kan hantera både låg och hög belastning utan manuell konfiguration.

2. **Flexibel och skalbar datalagring**

Valet av DynamoDB introducerade utmaningen att modellera dynamisk användardata utan ett fast schema. Jag implementerade en NoSQL-struktur som stödjer frekventa uppdateringar av matlistor med bibehållen hög prestanda och automatisk skalning.

3. **Enhetsbaserad autentisering**

Utan ett inloggningssystem var det en utmaning att identifiera användare säkert. Jag löste detta genom att generera ett unikt Device ID (UUID) vid appens första uppstart och lagra det lokalt. Varje API-förfrågan använder detta ID för att validera och isolera användardata, vilket säkerställer sekretess och persistens per enhet.

4. **AI-drivna måltidsförslag**

Integrationen med AWS Bedrock krävde att jag designade effektiva promptar och hanterade AI-svar på ett bra sätt. Detta gör att appen kan ge personliga och relevanta förslag som förbättrar användarupplevelsen och ökar variationen av maträtter.

5. **Separering av mikrotjänster**

Backend är uppbyggd av individuella Lambda-funktioner enligt

mikrotjänst-principer. Varje funktion har ett eget ansvar, vilket förbättrar underhåll och förhindrar beroendeproblem mellan funktionerna.

6. **Säker konfigurationshantering**

Känsliga uppgifter och miljövariabler lagras i AWS Systems Manager Parameter Store. Detta förhindrar hårdkodning av hemligheter och stödjer säker deployment.

7. **API-design**

RESTful API-design:

API:et följer REST-arkitekturprinciper:

- **Statelessness:** Varje förfrågan innehåller all nödvändig information (device ID), ingen sessionslagring på servern
- **Separation mellan klient och server:** Mobilfrontend och serverless-backend är helt frikopplade
- **Enhetligt gränssnitt:** Standardiserade HTTP-metoder (GET, POST, PUT, DELETE) och statuskoder

8. **Separation of Concerns:**

Varje Lambda-funktion följer Single Responsibility Principle och hanterar en specifik operation. Detta förbättrar underhållbarhet och testbarhet.

2.3 Arkitekturval

Serverless-arkitektur

Serverless-arkitektur valdes för detta projekt. Serverless är ett arkitekturmönster som gör det möjligt att utveckla och driftsätta applikationer utan att hantera traditionell serverinfrastruktur. Molnleverantören hanterar automatiskt provisionering, skalning och underhåll av resurser. I detta projekt används AWS Lambda för att köra backend-kod vid behov, medan Amazon API Gateway hanterar API-kommunikationen mellan frontend och backend.

Serverless följer en händelsedrivna modell, vilket innebär att kod endast körs när specifika händelser inträffar — till exempel när mobilappen anropar ett API för att hämta användarens matlista.

De viktigaste fördelarna med serverless-arkitektur i denna applikation är:

- **Snabb utveckling** — ingen serverkonfiguration eller serverunderhåll behövs
- **Automatisk skalning** — applikationen skalar direkt baserat på antalet användare

- **Kostnadseffektivitet** — betalning sker endast per exekvering, vilket gör det mycket prisvärt för en liten eller växande applikation

Dock finns det vissa utmaningar med serverless:

- **Cold starts** — om en funktion inte används på ett tag kan den första körningen ta längre tid medan miljön startas upp
- **Begränsad kontroll** — utvecklare är beroende av molnleverantören för exempelvis kapacitet, runtime-uppdateringar och skalningsmekanismer

Trots dessa kompromisser är serverless mycket väl lämpat för detta projekt eftersom det erbjuder hög skalbarhet, låg kostnad och minskad operativ komplexitet — perfekt för en modern mobilapplikation med oförutsägbar trafik.

3. Verktyg och Teknologier

Frontend

React Native

React Native valdes eftersom jag redan har viss erfarenhet av React, vilket gör det möjligt att återanvända bekanta koncept som `useState` och `useEffect`. Utveckling för mobil var dock helt nytt för mig, vilket uppfyllde mitt mål att lära mig något nytt. En och samma kodbas kan användas för både Android och iOS, vilket är mycket praktiskt om applikationen i framtiden ska publiceras i App Store.

Expo

Expo är en samling verktyg och tjänster byggt ovanpå React Native. Det möjliggör snabb utveckling, enkel testning och en smidig byggprocess, vilket gör det idealiskt för nybörjare eller projekt som kräver snabb prototypframtagning. Jag använde det för att bygga utvecklingsmiljön och kunna ansluta till Android-emulatorn så att jag direkt kunde se resultatet av mitt arbete.

AsyncStorage

Används för lokal datalagring av device ID och annan persistensdata. I detta utvecklingsskede behövs inget inloggningssystem, så ett enhets-ID används istället och lagras på telefonen. Detta innebär att användaren inte behöver logga in igen vid nästa användning.

Axios

HTTP-klient för API-anrop. Den ger bättre felhantering och möjliggör interceptors för att automatiskt lägga till headers, vilket undviker hårdkodning vid varje API-anrop och ger en renare kodstruktur.

UUID

Används för att generera unika device ID:n. Valdes eftersom det är enkelt att använda och skapar ett permanent ID vid första installationen av applikationen.

Backend

C# .NET 8

Programmeringsspråk och ramverk för backend. Implementeras tillsammans med AWS Lambda och fungerar väl med AWS. Jag har dessutom tidigare erfarenhet av språket.

AWS Lambda

Serverless compute-tjänst. Lambda valdes eftersom det är enkelt att köra kod utan serverhantering och erbjuder autoskalning. I applikationen används Lambda för att utföra all affärslogik som utgör applikationens kärnfunktioner.

AWS DynamoDB

NoSQL-databas. Den är skalbar, snabb och kräver ingen fördefinierad datamodell (schema). Passar bra när användardata kan ändras ofta.

AWS IAM

Identity and Access Management. Ger säker åtkomst och principer för minsta privilegium. Används för att styra åtkomst till Lambda och andra AWS-tjänster.

AWS Systems Manager

Konfigurationshantering. Används för att lagra tabellnamn och regioner på ett säkert sätt. Ger centraliserad och säker lagring av konfigurationsvärden och autentiseringsuppgifter.

AWS CloudWatch

Övervakning och loggning. Används för felsökning och loggning av funktionernas exekvering.

AWS API Gateway

Hanterar REST API-tjänster. Gör det enkelt att skapa API:er och integrera dem med Lambda. Hanterar även HTTPS. Används i applikationen för att dirigera anrop till rätt Lambda-funktion.

AWS Bedrock

AI-tjänst som används för att ge menyförslag baserat på favoriter och kriterier. Tjänsten består av förtränade AI-modeller och är enkel att integrera.

Utvecklingsverktyg

POSTMAN

Används för API-testning. Man kan spara alla anrop i en samling, se visuellt hur API:er svarar och även skriva API-dokumentation direkt i verktyget. Valdes för att det är enkelt att använda och ger en tydlig visualisering av dataflöden.

Visual Studio Code

IDE som används för att köra och utveckla koden. Har bra stöd för både C# och React Native via extensions.

Android Emulator

Används för testning och förhandsvisning av UI. När komponenter ändras syns det direkt i emulatorn. Det går även att köra på en fysisk mobiltelefon med app-ikon synlig.

AWS Console

Används som ett visuellt gränssnitt för arbete med AWS-tjänster.

GIT/GITHUB

Versionshantering, backup och spårning av ändringar. GitHub valdes för att det är enkelt och gratis att lagra kod där.

Externa API:er och tjänster

TheMealDB API

Gratis API för matdata och bilder, med bra täckning.

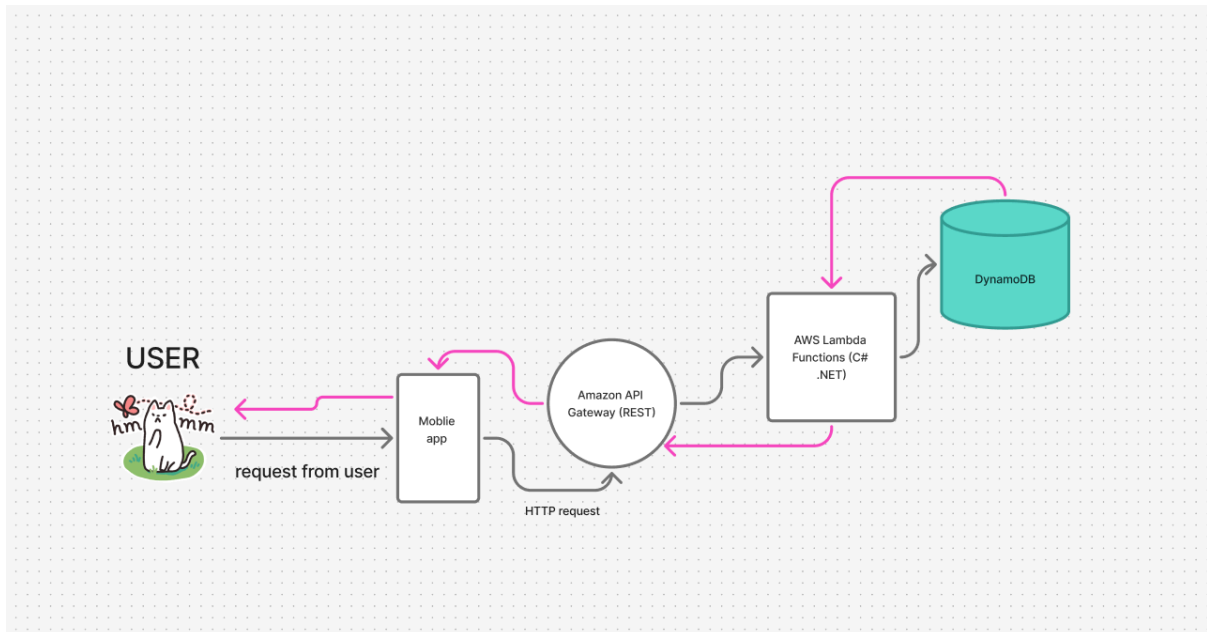
Pexels API

Används som fallback för matbilder om TheMealDB saknar bild som matchar maträtten.

4. Projektarkitektur

4.1 Systemöversikt

Applikationen implementerar en *serverlös mikrotjänstarkitektur* med AWS molntjänster, vilket möjliggör automatisk skalning, hög tillgänglighet och minskad operativ komplexitet. Systemarkitekturen illustreras i Figur 1 nedan.



[arkitektur](#)

Huvudkomponenter:

Systemet består av fem huvudsakliga komponenter som arbetar tillsammans:

1. **Mobilapplikation (React Native)** – Klientgränssnitt som hanterar användarinteraktioner och visar data. Kommunikerar med backend genom REST API-anrop.
2. **API Gateway** – Tjänsten fungerar som ingångspunkt för alla API-anrop. Hanterar HTTPS-endpoints, begäranderouting och vidarebefordrar anrop till rätt Lambda-funktioner.
3. **Lambda-funktioner** – Nio fristående serverlösa funktioner som exekverar backendens affärslogik. Varje funktion ansvarar för en specifik uppgift (CRUD-operationer, menygenerering, AI-förslag) enligt mikrotjänstprinciper.

4. **DynamoDB** – NoSQL-databas som lagrar maträtter och enhetsregistreringar. Ger snabb och skalbar åtkomst med automatisk replikering och backup.
5. **AWS Bedrock** – Hanterad AI-tjänst som används för att generera personliga matförslag och recept baserat på användarens preferenser.

Arbetsmetod för arkitekturen:

Designen följer Clean Architecture och mikrotjänstprinciper, där varje Lambda-funktion körs oberoende utan beroende till andra. Denna ansvarsfördelning ökar systemets effektivitet, underhållbarhet och gör det möjligt för komponenter att skalas individuellt baserat på belastning. Den händelsedrivna karaktären hos Lambda innebär att kod endast körs vid anrop, vilket ger kostnadseffektiv drift och automatisk skalning från noll till tusentals samtidiga begäranden.

Kommunikation mellan komponenterna följer standardiserade REST-mönster, där mobilapplikationen skickar HTTPS-anrop som går via API Gateway till Lambda-funktioner, som i sin tur interagerar med DynamoDB eller Bedrock vid behov.

4.2 Fil- och mappstruktur

Frontend-struktur (React Native Application)

```
RandomApp/
├── screens/                                ← All UI screens
│   ├── SplashScreen.jsx                  (Initial loading screen)
│   ├── Welcome.jsx                      (Welcome/onboarding)
│   ├── AddFood.jsx                      (Add new food form)
│   ├── FoodList.jsx                    (Display all saved foods)
│   ├── EditFood.jsx                    (Edit existing food)
│   ├── GenerateMenu.jsx                (Generate random menu)
│   ├── RecipeDetail.jsx                (Display recipe details)
│   └── Suggestions.jsx                  (AI food suggestions)
├── utils/                                ← Utility functions & helpers
│   ├── api.jsx                          (API communication with backend)
│   ├── deviceId.jsx                     (Device ID generation & storage)
│   ├── Helper.jsx                       (General helper functions)
│   ├── ImageHelper.jsx                  (Image URL generation & fallback)
│   └── colors.js                         (App color palette)
```

— assets/	← Images
— android/	← Android native build files
— .expo/	← Expo configuration
— node_modules/	← node Dependencies
— App.js	← Main app entry & navigation
— app.json	← Expo configuration
— package.json	← Dependencies & scripts
— babel.config.js	← Babel configuration
— .env	← Environment variables
— .gitignore	← Git ignore rules

Frontend-strukturen är organiserad för att stödja ren utveckling och enkel hantering av applikationsfunktioner. De huvudsakliga komponenterna är:

App.js

Detta är applikationens startpunkt. Den initierar navigation, global konfiguration och laddar nödvändiga resurser innan gränssnittet visas.

Assets-mapp

Denna mapp lagrar statiska filer såsom:

- Applikationsikoner
- Splash screen-bilder för Expo
- Lottie-animationer för onboarding och laddningsskärmar

Konfigurationsfiler

- **babel.config.js**: Säkerställer korrekt konfiguration för React Native-funktioner, såsom animationer
- **package.json**: Definierar projektets beroenden och scripts för bygg och körning
- **app.json**: Expo-konfiguration med appinställningar som namn, ikoner och splash screen

- **android/**: Genereras endast vid skapande av native Android-build och innehåller Gradle-filer och plattformskod

Mappstruktur

Applikationen är strukturerad i två huvudmappar:

1. utils/

Innehåller hjälplogik och återanvändbara funktioner:

- **colors.js** – central färgpalett för konsekvent UI-design
- **api.js** – kommunicerar med backend via Axios
- **deviceId.js** – genererar och lagrar en unik UUID per enhet
- **imageHelper.js** – hämtar matbilder från TheMealDB och fallback från Pexels

Separeringen förbättrar återanvändbarhet och minskar duplicerad kod.

2. screens/

Innehåller alla UI-skärmar som visas för användaren:

- **SplashScreen** – visas vid varje uppstart under initial laddning
- **WelcomeScreen** – visas endast vid första användning för att samla in användarnamn och lagra device ID
- Övriga skärmar för menygenerering, favoritlistor och matförslag

Varje skärm hanterar ett tydligt steg i användarflödet och underlättar modulär utveckling.

Principer för kodorganisation

- **Separation of concerns** – UI (screens) separeras från logik (utils)
- **Återanvändbarhet** – vanliga funktioner centraliseras i utils

- **Modularitet** – varje skärm är självständig
- **Konsekvent namngivning** – tydliga och beskrivande filnamn

Backendstruktur

```
RandomMenuLambda/
├── AddFoodFunction/
│   ├── src/AddFoodFunction/
│   │   ├── Function.cs
│   │   ├── AddFoodFunction.csproj
│   │   └── aws-lambda-tools-defaults.json
│   └── test/AddFoodFunction.Tests/
├── DeleteFoodFunction/
├── GetFoodsFunction/
├── UpdateFoodFunction/
├── GenerateMenu/
├── GetRecipeFunction/
├── SuggestNewFoodFunction/
├── SuggestByCriteriaFunction/
├── RegisterDeviceFunction/
└── Models/
    ├── RecipeResponse.cs
    ├── Recipe.cs
    ├── FoodDiscovery.cs
    ├── ConfigHelper.cs
    ├── DeviceHelper.cs
    ├── CriteriaSuggestion.cs
    ├── DeviceRegistration.cs
    ├── FoodItem.cs
    └── Models.csproj
```

← Shared Library Project

(Helper utilities)

(Helper utilities)

(Data models)

Backend följer en serverlös mikrotjänstarkitektur där varje Lambda-funktion är självständig och ansvarar för en specifik uppgift. Gemensam logik hanteras genom ett delat bibliotek vilket säkerställer kodåteranvändning och konsekvens mellan funktionerna.

Mappstruktur

Backend är organiserad i två huvudområden: **källkod** och **enhetstester**.

Källkodsstruktur

Varje Lambda-funktion följer en konsekvent struktur:

```
src/[FunctionName]/
```

- **Function.cs**
Innehåller Lambda-handlern och kärnan i affärslogiken.
- **[FunctionName].csproj**
Projektfil med beroenden och referenser till det delade Models-biblioteket.
- **aws-lambda-tools-defaults.json**
Deploy-konfiguration som används för att publicera Lambda-funktionen till AWS.

Denna struktur säkerställer att varje funktion är modulär och kan deployas oberoende.

Enhetstester

```
tests/[FunctionName].Tests/
```

Varje funktion har ett motsvarande testprojekt för att validera och säkerställa dess pålitlighet. Detta förbättrar utvecklingskvalitet och förenklar felsökning.

Delat kodbibliotek

Ett separat delat bibliotek kallat **Models** refereras av alla Lambda-funktioner.

Biblioteket innehåller:

- **Datamodeller** – representerar enheter såsom användare och maträtter
- **Utility Helpers** – exempelvis validering av Device ID och konfigurationshantering

- **Konstanter och konfiguration** – förhindrar hårdkodning och förbättrar underhållbarhet

Genom att centralisera gemensamma komponenter uppnår backend en bättre struktur, enklare uppdateringar och minskad kodduplicering.

5. Applikationens Komponenter

Backend componenter

Backendens består av 9 Lambda-funktioner som exponeras genom Amazon API Gateway som ett RESTful API. Varje funktion triggas av specifika HTTP-endpoints som hanteras av API Gateway, vilket sköter routning av förfrågningar, HTTPS-terminering och vidarebefordrar requests till lämplig Lambda-funktion. Varje funktion utför en specifik operation och inkluderar ägarskapsvalidering med deviceId för att säkerställa säkerhet och dataisolering.

```
API Endpoint (vad användaren anropar)
  ↓
Routas till
  ↓
Lambda-funktion (vad som exekveras)
```

AddFoodFunction

API Endpoint: `POST /foods`

Syfte: Skapar nya maträtter i användarens samling

Input:

- Request body: {deviceId, FoodName, foodId}
- Headers: x-device-id

Process:

1. Hämtar foods-tabellnamn från ConfigHelper
2. Validerar att deviceId och FoodName inte är null eller tomma
3. Trimmer whitespace och validerar längder
4. Verifierar att enheten är registrerad med DeviceHelper
5. Genererar unikt foodId
6. Ansluter till DynamoDB-klient
7. Sparar maträtt till Foods-tabell
8. Returnerar success-response med skapad maträtt

Output:

- Success: 200 OK med skapad maträtt
- Errors: 400/404/500 med felmeddelanden

GenerateMenuFunction

API Endpoint: GET /menu

Syfte: Genererar slumpmässig meny baserad på användarens favoriträtter

Input:

- Request body: {menu: randomFood, generatedAt}
- Headers: x-device-id

Process:

1. Hämtar foods-tabellnamn från ConfigHelper
2. Extraherar deviceId från query-parametrar
3. Validerar deviceId (inte tom, korrekt längd)
4. Trimmar whitespace
5. Kontrollerar om enheten är registrerad
6. Skapar DynamoDB-klientanslutning
7. Kör query i DynamoDB för alla maträtter för deviceId
8. Exekverar query och konverterar till List<FoodItem>
9. Kontrollerar om listan är tom (minimum 3 maträtter krävs)
10. Väljer slumpmässigt en maträtt från varje kategori
11. Returnerar response med genererad meny

Output:

- Success: 200 OK med menyobjekt innehållande frukost, lunch, middag
- Errors: 400 om inte tillräckligt med maträtter, 404 om enheten inte är registrerad

SuggestNewFoodFunction

API Endpoint: GET /suggest-food

Syfte: Genererar AI-drivna matförslag baserade på användarens favoriträtter med AWS Bedrock

Input:

- Request body: {SuggestedFood, Reason, Recipe, BasedOnFavorites}
- Headers: x-device-id

Process:

1. Hämtar suggestion history-tabellnamn från ConfigHelper
2. Extraherar deviceId från query-parametrar
3. Trimmar whitespace och validerar att deviceId existerar
4. Hämtar foods-tabellnamn från ConfigHelper
5. Skapar DynamoDB-klient
6. Kör query för alla användarens favoriträtter
7. Exekverar query och extraherar lista med favoritnamn
8. Beräknar datum 7 dagar tillbaka för historik
9. Kör query för tidigare förslag för att undvika upprepning
10. Exekverar query och extraherar lista med tidigare förslag
11. Hanterar fall om användaren inte har några sparade maträtter
12. Bygger exkluderingstext för AI (undvik att föreslå samma rätter)
13. Konstruerar AI-systemprompt som definierar assistentens roll
14. Konstruerar AI-användarprompt med favoriter och exkluderingar
15. Bygger Bedrock InvokeModelRequest med Claude 3 Haiku
16. Anropar Bedrock API (kallar AI-tjänst)
17. Läser response-stream från Bedrock
18. Konverterar stream till string
19. Parsar Bedrock response wrapper-JSON
20. Extraherar content-array från response
21. Hämtar första content-item (AI:s förslag)
22. Parsar föreslagen maträtt, anledning och recept från JSON
23. Bygger FoodDiscovery response-objekt
24. Sparar nytt förslag till suggestion history-tabell med TTL
25. Exekverar PutItem-request för att lagra historik
26. Returnerar success-response med AI-förslag och recept

Output:

- Success: 200 OK med matförslag, anledning och komplett recept
- Errors: 400 om inga maträtter sparade, 500 om AI-parsning misslyckas

Övriga Lambda-funktioner

Följande funktioner tillhandahåller stödjande CRUD-operationer:

RegisterDeviceFunction

Endpoint: `POST /register`

Registrerar en ny enhet när appen startas första gången. Sparar device ID och användarnamn till DeviceRegistry-tabell. Returnerar 200 om registreringen lyckades, eller 409 om enheten redan existerar.

GetFoodsFunction

Endpoint: `GET /foods/{deviceId}`

Hämtar alla maträtter som användaren har sparat. Kör query i DynamoDB med deviceId och returnerar komplett maträttslista. Returnerar 200 med array av maträtter, eller 404 om inga maträtter hittades.

UpdateFoodFunction

Endpoint: `PUT /foods/{deviceId}/{foodId}`

Uppdaterar en befintlig maträtts namn eller kategori. Validerar enhetsägarskap innan modifiering tillåts. Returnerar 200 med uppdaterat maträttsobjekt, eller 404 om maträtten inte hittades.

DeleteFoodFunction

Endpoint: `DELETE /foods/{deviceId}/{foodId}`

Raderar en specifik maträtt från användarens lista. Validerar att enheten äger maträtten innan radering. Returnerar 200 om raderingen lyckades, eller 404 om maträtten inte hittades.

GetRecipeFunction

Endpoint: `GET /recipe/{foodName}`

Hämtar ett detaljerat recept för en vald maträtt med hjälp av AI. Returnerar recept med ingredienser, instruktioner, tillagningstid och svårighetsgrad.

SuggestByCriteriaFunction

Endpoint: `GET /suggest-by-criteria/{deviceId}/{criteria}`

Genererar matförslag baserat på användarspecificerade kriterier som "kryddigt", "thailändskt" eller "vegetariskt". Använder AWS Bedrock AI med kriteriebaserade prompter. Undviker att föreslå maträtter som visats under den senaste veckan.

Databas

Systemet använder Amazon DynamoDB som primär datalagring. Databasen består av tre tabeller, var och en designad för att stödja viktiga applikationsfunktioner samtidigt som effektiv frågning och skalbarhet säkerställs.

1. Foods-tabell

Denna tabell lagrar alla maträtter som läggs till av användare.

Syfte:

- Lagra användardefinierade favoriträtter
- Stödja fullständiga CRUD-operationer för menyhantering
- Möjliggöra slumpmässig menygenerering genom att fråga alla maträtter per enhet

8211cb84-5a60-4669...	ef844b02-2eee-4bb3...	pizza hawaii
test-device-register-789	1cd4e09e-17d1-48c8...	pizza with apple

2. DeviceRegistry-tabell

När en användare startar applikationen första gången genereras ett unikt UUID och lagras tillsammans med användarens namn i denna tabell. Detta skapar en persistent identifierare som gör det möjligt för applikationen att känna igen samma användare på samma enhet utan inloggningsfunktionalitet.

Syfte:

- Registrera nya enheter vid första appstart
- Associera ett användarvänligt namn med varje unik enhet
- Möjliggöra enhetsbaserad autentisering utan lösenord

- Validera enhetsägarskap för alla API-förfrågningar

8211cb84-5a...  	2025-12-0...	2025-12-0...	queen may
test-device-register-789	2025-11-1...	2025-11-1...	maya

3. FoodSuggestionHistory-tabell

Denna tabell lagrar AI-genererade matförslag som visats för användare. Varje post inkluderar föreslagen maträttsnamn, tidsstämpel och använda kriterier. En dataretentionspolicy raderar automatiskt poster äldre än 7 dagar med DynamoDB TTL (Time To Live) för att optimera lagringskostnader och bibehålla databas effektivitet.

Syfte:

- Spåra senaste AI-genererade förslag per enhet
- Förhindra repetitiva förslag inom 7-dagarsfönster
- Tillhandahålla korttidshistorik för förbättrade AI-rekommendationer
- Automatiskt städa bort gammal data för kostnadsoptimering

8211cb84-5a60-4669...	2025-12-01T21:18:38.18781...	spicy	Sichuan cui...	Sichuan Chongq...	criteria
8211cb84-5a60-4669...	2025-12-01T21:50:37.47328...	[{"S": "pad thai"}, ...	Based on th...	Korean Spicy Po...	Favorite

Frontend componenter

Fronten inkluderar åtta skärmar, var och en ansvarig för olika användarinteraktioner inom applikationen:

SplashScreen

En animerad startskärm som visas när applikationen startar. Den använder en Lottie-animation lagrad som en JSON-fil, kombinerad med Expos animationsverktyg. Detta skapar en smidig visuell laddningsupplevelse innan huvudgränssnittet blir tillgängligt.

```
useEffect(() => {
  const timer = setTimeout(() => {
    Animated.timing(animationRef, {
      toValue: 0,
      duration: 550,
      useNativeDriver: true,
    })
  }, 500)
})
```

```

        }).start(() => {
            onFinish();
        });
    }, 5000);
    return () => clearTimeout(timer);
}, [onFinish]);

<Animated.View style={[styles.container, { opacity:
animationRef }]}>
    <LottieView
        source={splashScreenAnimation}
        style={styles.animation}
        autoPlay
        loop={false}
        resizeMode="cover"
    />
</Animated.View>

```

WelcomeScreen

Visas endast första gången användaren öppnar applikationen. Ett unikt UUID genereras och sparas lokalt tillsammans med användarens namn. Vid framtida starter hoppar appen över denna skärm och navigerar direkt till Hem-skärmen eftersom enheten redan är registrerad.

AddFoodScreen

Låter användare lägga till sina favoriträtter. När användaren trycker på Lägg till-knappen visas en modal med animation där användaren anger maträttsnamn och väljer kategori. En bekräftelsepopup visas när maträtten framgångsrikt sparats till databasen.

FoodListScreen

Visar användarens hela sparade maträttslista organiserad efter kategori. Användare kan lägga till nya maträtter, redigera befintliga maträtter eller radera maträtter direkt från denna skärm. Inkluderar pull-to-refresh-funktionalitet för synkronisering med backend.

EditFoodScreen

Gör det möjligt för användare att ändra namn eller kategori på en maträtt genom att trycka på redigera-ikonen (penna). Användare kan antingen Spara den uppdaterade informationen eller Avbryta om de ändrar sig. Ändringar synkroniseras omedelbart till DynamoDB.

GenerateMenuScreen

Genererar en slumpmässig dagsmeny genom att välja en maträtt från varje kategori (frukost, lunch, middag). För att aktivera denna funktion måste användare ha minst tre sparade favoriträtter (en per kategori). När villkoret är uppfyllt kan en komplett meny genereras med ett enda tryck.

RecipeDetailScreen

Visar detaljerade recept för föreslagna rätter, inklusive ingredienslista, steg-för-steg-instruktioner, förberedelsetid, tillagningstid och svårighetsgrad. Särskilt användbar när användaren väljer att laga en föreslagen måltid hemma.

SuggestionsScreen

Tillgänglig efter att en meny har genererats. En modal visas som låter användaren välja mellan två förslagstyper:

- **Baserat på favoriter** - AI analyserar användarens befintliga maträtter och föreslår liknande rätter
- **Baserat på kriterier** - Användaren anger söktermer som "kryddigt", "thailändskt" eller "vegetariskt"

Användare kan visa föreslagna rätter med kompletta recept och spara dem direkt till sin favoritlista.

Interna Moduler

Återanvändbara kodmoduler implementeras i både backend och frontend för att säkerställa konsistens, undvika duplicering och förbättra underhållbarhet.

Backend Delade Moduler

Två delade hjälpmoduler inkluderas i backend Models-biblioteket och refereras av alla Lambda-funktioner:

ConfigHelper

Syfte: Hämtar konfigurationsvärden från AWS Systems Manager Parameter Store

Nyckelmetoder:

- `GetFoodsTableNameAsync()` - Returnerar DynamoDB foods-tabellnamn
- `GetDeviceRegisterTableNameAsync()` - Returnerar enhetsregistreringstabell-namn
- `GetSuggestionHistoryTableNameAsync()` - Returnerar förslagshistoriktabell-namn

Användning: Alla Lambda-funktioner anropar ConfigHelper för att hämta tabellnamn istället för att hårdkoda dem, vilket möjliggör enkla konfigurationsändringar utan kodomployering.

DeviceHelper

Syfte: Hanterar enhetsregistrering och validering

Nyckelmetoder:

- `IsDeviceRegisteredAsync(string deviceId)` - Kontrollerar om enheten existerar i DeviceRegistry-tabell
- `GetDeviceInfoAsync(string deviceId)` - Returnerar enhetsinformation
- `CreateUnregisteredDeviceResponse()` - Returnerar standard 403-felsvar

Användning: Lambda-funktioner anropar DeviceHelper innan bearbetning av förfrågningar för att säkerställa att enheten är registrerad och har behörighet att komma åt data.

Exempelkod:

```
public static APIGatewayProxyResponse CreateUnregisteredDeviceResponse()
{
    return new APIGatewayProxyResponse
    {
        StatusCode = 403,
        Body = JsonSerializer.Serialize(new { error = "Device is not
registered" }),
        Headers = new Dictionary<string, string>
        {
            { "Content-Type", "application/json" }
        }
    };
}
```

Frontend Utility-bibliotek (utils/)

Fronten inkluderar ett dedikerat internt hjälpbibliotek för att organisera kärnfunktionalitet:

api.js

Hanterar all kommunikation med backend via Axios. Använder interceptors för att automatiskt inkludera headers som device ID. Förbättrar felhantering och undviker upprepad kod i flera komponenter.

imageHelper.js

Hämtar matbilder från TheMealDB som primär källa. Faller automatiskt tillbaka på Pexels API om primärkällan saknar lämplig bild. Säkerställer en konsekvent visuell upplevelse för alla maträtter.

helper.js

Innehåller generiska hjälpfunktioner som dynamiska hälsningsmeddelanden baserade på tid på dygnet och lagrat användarnamn. Tillhandahåller återanvändbara hjälpmetoder som används över flera skärmar.

deviceId.js

Genererar ett unikt UUID vid första applikationsstarten med React Natives AsyncStorage. Lagrar och hämtar det persistenta device ID:t för framtida appsessioner. Stödjer backendssystemet för enhetsbaserad användaridentifiering.

colors.js

Centraliserad färgpalett som används genom hela appen. Säkerställer konsekvent UI-styling och undviker hårdkodade färgvärden i enskilda skärmar.

Exempelanvändning:

```
color: COLORS.textDark,  
backgroundColor: COLORS.cardBackground,
```

6. Flöden

Autentisering

Applikationen använder enhetsbaserad autentisering istället för traditionell användarnamn/lösenord-inloggning. När en användare startar appen första gången genereras ett unikt UUID med hjälp av uuid-biblioteket och lagras lokalt i AsyncStorage.

Vid varje API-förfrågan inkluderas device ID i x-device-id-headern. Lambda-funktionerna validerar device ID genom att kontrollera om det existerar i DeviceRegistry-tabellen innan förfrågan bearbetas. Om enheten inte är registrerad returnerar API:et ett 404-fel.

Fördelar:

- Enkelt för användare med en enhet
- Ingen lösenordshantering krävs
- Varje enhet upprätthåller separata data
- Omedelbar åtkomst efter första starten

Begränsningar:

- Data är bunden till endast en enhet
- Avinstallation av appen medför förlust av dataåtkomst
- Ingen synkronisering mellan enheter (kan inte använda samma data på iPhone och iPad)
- Framtida förbättring kommer att använda AWS Cognito för korrekt multi-enhetsautentisering

Dataflöden

Alla dataflöden i applikationen följer ett konsekvent mönster: mobilappen skickar en HTTPS-förfrågan genom API Gateway till en specifik Lambda-funktion, som bearbetar förfrågan, interagerar med DynamoDB vid behov, och returnerar ett svar tillbaka genom samma väg. Varje förfrågan inkluderar device ID i headers för autentisering, och Lambda-funktionerna validerar detta innan bearbetning.

Systemet använder en request-response-modell där mobilappen väntar på att backend ska slutföra bearbetningen innan UI:et uppdateras. Nedan är ett detaljerat exempel på det kompletta dataflödet för att lägga till en maträtt.

```
Mobile App → API Gateway → Lambda Functions → DynamoDB (Router)
(Business Logic)
```

Exempel: Lägga till en maträtt (Adding a Food Item)

Detta demonstrerar den kompletta resan av data från användaråtgärd till databaslagring och tillbaka.

Steg-för-steg-flöde:

1. **Användaråtgärd:** Användaren öppnar AddFood-skärmen, anger "Pizza Margherita", väljer kategori "Dinner", trycker på "Add Food"
2. **Mobilapp (Frontend):**
 - Hämtar device ID från AsyncStorage: `8211cb84-5a60...`
 - Anropar `addFood()` funktion i `api.jsx`
 - Skickar POST-förfrågan till `https://mw12gj0shi.execute-api.eu-west-1.amazonaws.com/dev/foods`
 - Headers: `{ "x-device-id": "8211cb84-5a60...", "Content-Type": "application/json" }`
 - Body: `{ deviceId: 8211cb84-5a60..., "FoodName": "bulgogi biff" }`
3. **API Gateway:**
 - Tar emot HTTPS-förfrågan på `POST /foods endpoint`
 - Validerar förfrågningsformat
 - Routar till AddFoodFunction Lambda
 - Skickar headers och body till Lambda
4. **AddFoodFunction Lambda:**
 - Extraherar deviceId från request headers
 - Anropar `DeviceHelper.IsDeviceRegisteredAsync(deviceId)`
 - Validerar input (name inte tom)
 - Genererar unikt food ID: `50b148e2-e2ac...`
 - Bygger DynamoDB item-objekt
 - Anropar `dynamoDbClient.PutItemAsync()` för att spara
5. **DynamoDB:**
 - Tar emot PutItem-förfrågan för Foods-tabell

- Lagrar item med nycklar: deviceId + foodId
- Sparar attribut: FoodName
- Returnerar bekräftelse till Lambda

6. Lambda-svar:

- Skapar success-svarsobjekt
- Returnerar 200 OKstatus
- Body: `{ "foodId": "50b148e2-e2ac...", "name": "bulgogi biff" }`

7. API Gateway:

- Tar emot svar från Lambda
- Vidarebefordrar till mobilapp över HTTPS

8. Mobilapp (Frontend):

- Tar emot 200-svar
- Visar success-meddelande: "Food added successfully!"
- Rensar inputfält
- Användaren kan lägga till ytterligare maträtt eller navigera bort

Dataexempel vid varje steg:

```
Förfrågan: { "deviceId": "8211cb84-5a60-4669-90a4-019f14676cf6", "name": "bulgogi biff" }
```

```
DynamoDB item: {
  "deviceId": "8211cb84-5a60-4669-90a4-019f14676cf6",
  "foodId": "50b148e2-e2ac-4064-9817-84f32a11e420",
  "FoodName": "bulgogi biff"
}
```

```
Svar: { "deviceId": "8211cb84-5a60-4669-90a4-019f14676cf6", "foodId": "50b148e2-e2ac-4064-9817-84f32a11e420", "name": "bulgogi biff" }
```

Andra operationer (GetFoods, UpdateFood, DeleteFood, GenerateMenu, etc.) följer samma generella mönster med variationer i Lambda-bearbetningslogiken och databasoperationerna.

Externa API:er

Applikationen integrerar med externa tjänster för matbilder och AI-drivna förslag.

TheMealDB API

Primär källa för matbilder. Anropas från frontend ImageHelper.jsx när maträtter visas i applikationen. Om TheMealDB inte har önskad bild används Pexels som backup.

Pexels API

Sekundär bildkälla när TheMealDB saknar bilder. Tillhandahåller högkvalitativa matfoton som fallback-alternativ.

AWS Bedrock

AI-tjänst för att generera personaliserade matförslag och recept. Anropas från Lambda-funktioner (SuggestNewFoodFunction, SuggestByCriteriaFunction) med AWS SDK. Använder Claude 3 Haiku-modellen för snabba, kostnadseffektiva AI-svar.

7. Testning

Testningsstrategi

Testningen fokuserade på API-funktionalitet och end-to-end användarflöden för att säkerställa att applikationen fungerar korrekt i verklig användning. Testningsmetoden prioriterade integrationstestning och manuell testning framför enhetstestning på grund av projektets akademiska omfattning och tidslinje.

Testmetodik

API-testning (Postman):

Alla Lambda-funktioner testades med Postman för att verifiera korrekt beteende, korrekt felhantering och datavalidering. Varje endpoint testades med:

- Giltig indata
- Ogiltig indata (saknade fält, felaktiga typer)
- Saknade autentiseringsheaders
- Kantfall (tomma listor, specialtecken)

Manuell testning (Mobilapp):

Mobilapplikationen testades på Android-emulator och fysisk enhet för att verifiera:

- UI-funktionalitet och navigation
- API-integration och datapersistens
- Användarflöden från start till slut
- Felmeddelanden visas korrekt

Integrationstestning:

Kompleta användarflöden testades för att säkerställa att alla komponenter fungerar tillsammans:

- Enhetsregistrering → Lägg till maträtter → Generera meny
- Lägg till maträtter → Redigera maträtter → Ta bort maträtter
- Få AI-förslag → Visa recept

Testresultat

All kritisk funktionalitet godkände testning:

- Alla 9 Lambda-funktioner returnerar korrekta svar
- Device ID-autentisering fungerar över alla endpoints
- CRUD-operationer lagrar och hämtar framgångsrikt data från DynamoDB
- AI-förslag genererar giltiga matrekommendationer och recept
- Bildfallback-system fungerar (TheMealDB → Pexels)
- Felhantering returnerar lämpliga statuskoder och meddelanden
- Mobilappen kommunicerar framgångsrikt med backend

Kända begränsningar:

- Ingen automatiserad testsvit (endast manuell testning)
- Ingen belastningstestning utförd
- Testning gjord med begränsade dataset (10-20 maträtter per enhet)

8. Deployment

Körning och Deployment

Backend (Lambda-funktioner):

Lambda-funktioner deployeras manuellt till AWS med hjälp av AWS Toolkit i VS Code och aws CLI. Varje funktion byggs lokalt med dotnet build, paketeras och deployeras självständigt. Efter deployment routar API Gateway automatiskt förfrågningar till de uppdaterade funktionerna. Deployment verifieras genom CloudWatch-logga.

```
#create new lambda function
dotnet new lambda.EmptyFunction -n [Functionname]
# Deploy
dotnet lambda deploy-function [Functionname] --function-role
roles that you created.
```

Frontend (Mobilapp):

Mobilapplikationen körs i utvecklingsläge med Expo. På grund av beroendekrav (React Native Reanimated) använder appen Expo development builds istället för Expo Go:

```
npx expo run:android    # Development build på Android
npx expo start          # Starta development server
```

Appen ansluter direkt till produktions-AWS-backend under utveckling och testning.

Framtida produktionsdeployment:

För app store-deployment skulle applikationen kräva:

- EAS Build för att generera produktions-APK/AAB (Android) och IPA (iOS)
- Utvecklarkonton för Google Play Store (25 USD) och Apple App Store (99 USD/år)
- App store-efterlevnad (integritetspolicy, skärmdumpar, metadata)

Miljövariabler och Konfiguration

Backend-konfiguration (AWS Systems Manager Parameter Store):

Alla konfigurationsvärden lagras säkert i Parameter Store:

- `/random-menu-app/table-name` - DynamoDB foods-tabellnamn
- `/random-menu-app/DeviceRegistryTableName` - Enhetsregistreringstabell-namn
- `/random-menu-app/suggestiontable` - Förslagshistoriktabell-namn

Lambda-funktioner hämtar dessa värden vid körning med ConfigHelper, vilket förhindrar hårdkodad konfiguration och möjliggör enkla uppdateringar utan omdeployering.

Frontend-konfiguration:

Miljövariabler lagras i `.env`-fil (committas ej till Git):

```
EXPO_PUBLIC_PEXELS_API_KEY: pexels api key.
```

Fördelar:

- Ingen känslig data i källkod
- Enkla konfigurationsändringar

Future Enhancements

1. AWS Cognito + JWT-autentisering

Byt från enhets-ID till inloggningssystem med full integration med JWT för autentisering baserad på API-design för säkerhet. Användare kan använda appen på iPad, telefon och dator - inte begränsat till endast mobila enheter. Ett konto kan logga in på flera enheter.

2. Receptlagring

Användare kan spara AI-genererade recept till databasen för att laga senare. Genom att klicka på en maträtt i sin matlista visas det sparade receptet med fullständiga detaljer.

3. Bildigenkänning

Istället för att skriva in maträtter en efter en kan användare ta ett foto av en meny eller recept. AI läser bilden och genererar menyförslag baserat på innehållet i bilden.

4. Restaurangsökning

Bäst för kontorsarbetare som genererar en meny och behöver hitta restauranger i närheten. Om restauranger är för långt bort föreslår appen alternativ: generera en ny meny, beställ via matleveranstjänster eller laga mat hemma.

5. Näringsstatistik

Funktion för personer som behöver gå ner i vikt eller kontrollera sitt matintag. Varje maträtt visar detaljerad näringsinformation inklusive kalorier, protein, kolhydrater och fett per måltid.

6. Viktminsknings-måltidsplanering

AI genererar personliga menyer baserat på användarens kaloriemål och viktminskningmål, vilket säkerställer att måltiderna håller sig inom dagliga kalorigränser samtidigt som näringsbalansen bibehålls.

9. Referenser

React Native. (2025). Getting-started.

<https://reactnative.dev/docs/getting-started>

Expo. (u.å). Expo Documentation.

<https://docs.expo.dev/>

<https://medium.com/@shiva999421/animating-with-lottie-simple-like-never-before-with-react-native-f32f165db0a5>

Expo. (2025, April 16). lottie-react-native.

<https://docs.expo.dev/versions/v53.0.0/sdk/lottie/>

Expo. (2025, Augsti 27). react-native-reanimated.

<https://docs.expo.dev/versions/latest/sdk/reanimated/>

aws lambda

Amazon Lambda Document. (2025). AWS Lambda Documentation.

https://docs.aws.amazon.com/en_us/lambda/

Amazon Lambda Document. (2025). Invoking Lambda with events from other AWS services.

<https://docs.aws.amazon.com/lambda/latest/dg/lambda-services.html>

Solo.io. (2025). AWS API gateway.

<https://www.solo.io/topics/api-gateway/aws-api-gateway>

geeksforgeeks. (2025, Juli 23). Amazon Web Service- Introduction to API Gateway.

<https://www.geeksforgeeks.org/devops/amazon-web-service-introduction-to-api-gateway/>

geeksforgeeks.(2025, November 21).DynamoDB - Introduction. geeksforgeeks.(2025, November 21).

<https://www.geeksforgeeks.org/python/dynamodb-introduction/>

geeksforgeeks.(2025, Juli 12). DynamoDB: Understand The Benefits With Real Life Use Cases.

<https://www.geeksforgeeks.org/blogs/dynamodb-understand-the-benefits-with-real-life-use-cases/>

Xu, Alex. (2023, Maj 10). Mastering the Art of API Design.

<https://blog.bytebytego.com/p/api-design>

Kumar, Suneel. (2024, Maj 13). API Design: From Basics to Best Practices.

<https://medium.com/@techsuneel99/api-design-from-basics-to-best-practices-da47c63aaf70>